

Reading Support: Computing Admissible Strings II

Companion to Week 10

Cem Bozsahin

LING487 · Fall 2026

A finite automaton has no stack

Supports: Finite-state computation remembers only a finite abstraction of the past

A finite automaton $M = (Q, \Sigma, \Delta, s, F)$ has finitely many states, an input alphabet, a transition relation, a start state, and accepting states. Its configuration $[q, \alpha]$ records only the current state and unread input.

The state must summarize every fact about the consumed prefix that can affect acceptance of a continuation.

MOLR §5.2

States preserve distinctions required by future continuations

Supports: A state is an equivalence class of relevant histories

Different prefixes may reach the same state when every possible continuation has the same acceptance consequence after either prefix. Conversely, prefixes must remain in different states when some continuation distinguishes them.

A state therefore represents a finite equivalence class of histories, not a copy of the material already consumed.

MOLR §5.2

Parity requires four combinations

Supports: Example: odd numbers of both *a*'s and *b*'s

*To recognize strings with odd numbers of both symbols, the machine must distinguish even-even, odd-even, even-odd, and odd-odd prefixes. Reading *a* changes the first parity; reading *b* changes the second.*

Only the odd-odd state accepts. The state labels express the invariant needed for every possible continuation.

MOLR §5.2

Nondeterminism admits several next configurations

Supports: Determinism is a property of the transition relation

A DFA has exactly one transition for every state-symbol pair. An NFA may have none, one, or several, and may move on ϵ . It accepts when at least one complete path reaches an accepting configuration.

Nondeterminism enlarges the set of possible configurations during a computation, but it does not enlarge the family of languages recognized by finite automata.

MOLR §5.2

Subset construction makes all NFA possibilities one DFA state

Supports: A DFA state can represent a set of NFA possibilities

The empty closure of a state contains every state reachable through ϵ -moves, including the state itself. For an input symbol, collect all destinations reachable from the current set, with empty closure before and after the consumed symbol.

Each distinct reachable set becomes one DFA state. It accepts if it contains any accepting NFA state.

MOLR §5.4

The deterministic simulation preserves acceptance

Supports: Subset construction preserves acceptance, not the original path story

After reading a prefix, the subset state records exactly the NFA states that could then be active. The DFA has one path, but that path carries the set of alternatives formerly distributed across many NFA paths.

Unreachable subsets need not be included. The conversion preserves the language without preserving the explanatory interpretation of individual NFA paths.

MOLR §5.4

Regular grammar continuation becomes state transition

Supports: Regular grammars and finite automata encode the same stringsets

In converting a right-linear grammar, make each nonterminal a state. A rule $A \rightarrow aB$ becomes a transition from A to B on a . A terminal rule ends in a designated accepting state.

The remaining nonterminal in a sentential form and the current automaton state encode the same question: what continuations are still admissible?

MOLR §5.4

State elimination combines all routes through a state

Supports: Automaton-to-regexp conversion eliminates internal states

A relaxed automaton labels an edge with a regular expression. When an internal state is removed, each incoming expression combines with its loop, if any, and each outgoing expression. Parallel routes join by union.

The resulting expression preserves every start-to-accept path while suppressing the intermediate state structure.

MOLR §5.4

A transducer labels paths on two tapes

Supports: A finite-state transducer computes a regular relation

A finite-state transducer places an input-output pair on each transition. Concatenating the first components along an accepting path gives an input string; concatenating the second gives a related output string.

Alignment with ϵ expresses insertion or deletion. Several accepting paths for one input can give several outputs unless the relation satisfies functionality.

MOLR §5.4

Composition hides a compatible intermediate representation

Supports: Composition makes finite-state analyses modular

If R_1 relates Σ^ to Γ^* and R_2 relates Γ^* to Δ^* , their composition relates an input to an output whenever some intermediate Γ -string connects the two relations.*

The same domain condition governs the function composition introduced at the start of the course.

MOLR §5.4